How PickNet Works

A step-by-step guide to the architecture

Companion to *PickNet: One Transformer for Human Draft Picks*

October 2026

The paper's Figure 1 squeezes the whole model into six boxes. This guide unpacks them one at a time. It assumes you know how a booster draft works and roughly what a neural network is (a function with lots of tunable numbers, fitted by gradient descent). It does not assume you know transformers. Every claim about the model refers to the code in `src/openlane/picknet.py`, and the sizes are those of the final all-sets model (`results/picknet_all/models/all_d256_ep2.pt`).

Contents

1 The job, and the big picture **2**

2 What one training example looks like **2**

3 Step 1: turning a card into a vector **4**

4 Step 2: building one history token per pick **5**
4.1 "What the others took" 6
4.2 Draft context 6

5 Step 3: the history encoder **7**
5.1 Attention in one paragraph 7
5.2 The causal mask 7

6 Step 4: candidate tokens **7**

7 Step 5: the pack decoder **8**

8 Step 6: from scores to a pick **8**

9 The deck head: a training aid **9**

10 Training **10**

11 Shapes and sizes **10**

12 Putting it together: one pick, start to finish **11**

13 FAQ **11**

1 The job, and the big picture

At every pick, PickNet looks at everything the drafter could have known at that moment and outputs a probability for each card in the pack: "the human takes card A with 57%, card B with 26%, …". It is trained to put as much probability as possible on the card the human actually took.

The model has two halves, and almost everything else is plumbing around them:

1. **The history encoder** reads the draft so far, one token per pick, and produces a *state* for each pick: a list of 256 numbers that summarises "what this drafter knows and is doing right now": their colours, their curve, what seems to be flowing, how strong a player they are.
2. **The pack decoder** takes the cards in the current pack, lets them look at each other and at the history, and gives each card a score. A softmax turns the scores into probabilities.

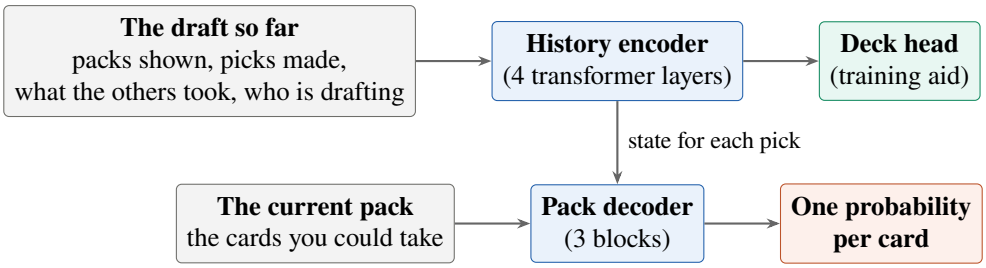


Figure 1: The model at the coarsest level. The deck head is only used during training; it gives the history encoder an extra thing to learn (which picks will make the deck).

A useful way to think about it: the history encoder answers "who am I as a drafter right now?" and the pack decoder answers "given that, which of these cards do I want, considering what else is in the pack?".

Figure 2 (page 3) is the same model with nothing left out: every layer that has trained weights, how many weights each one has, and the shape of every tensor for one draft. **Blue** boxes have trained weights; the small blue number is how many. **Grey** boxes are data, **white** boxes are fixed operations with no weights, ⊕ adds vectors, and ♦ marks a lookup in the card table. Shapes are per draft (a batch adds a leading B). The history-encoder layer is stacked 4 times and the pack-decoder block 3 times, each copy with its own weights; the thin grey lines that skip around each attention and MLP are the residual connections. Every decoder block reads the same history states **h**. Dropout (0.05) after each attention and MLP is not drawn. With the 65,792-weight pool layer that only the no-history ablation uses, the counts add up to 8,357,889. The rest of this guide walks through the figure from top to bottom, so it's worth keeping that page open while reading.

2 What one training example looks like

One training example is one *whole draft* by one drafter, not one pick. Every draft is laid out on a fixed grid of 3 packs × 15 pick slots = **45 slots**, numbered $t = 0, \ldots, 44$. Sets with 14- or 13-card packs leave the last one or two slots of each pack empty; those slots are masked out (ignored) in the loss and in every metric.

For each slot t the data holds:

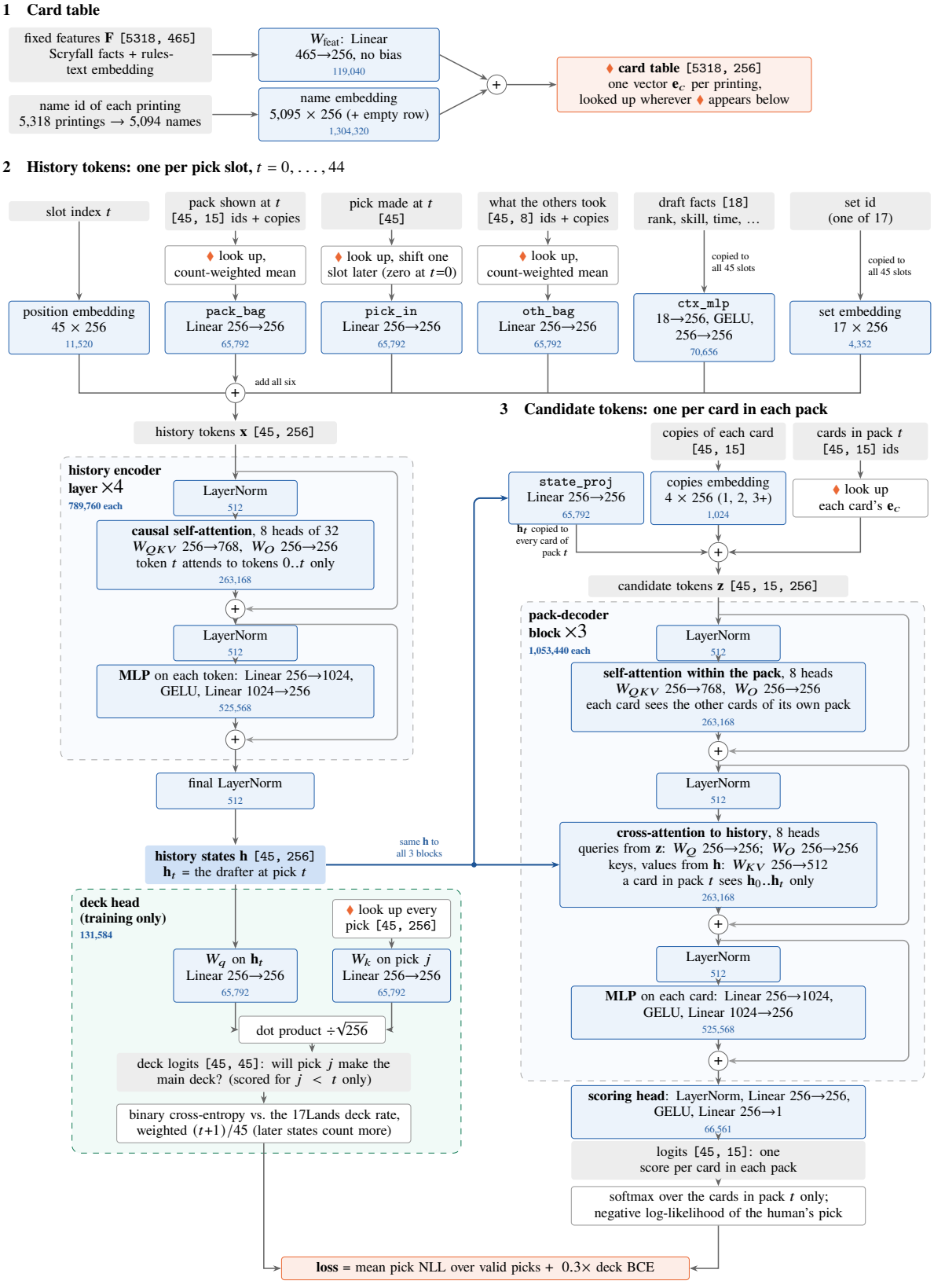


Figure 2: Every learned layer of PickNet, for one draft. Blue: trained weights (with counts). Grey: data. White: fixed operations. ♦: card-table lookup.

name in code	shape per draft	meaning
`idx`, `cnt`	`[45, 15]`	the pack shown at t: up to 15 distinct card ids, and how many copies of each
`pick`	`[45]`	the card the human took at t
`target`	`[45]`	which of the 15 `idx` slots that card is in (the answer)
`oidx`, `ocnt`	`[45, 8]`	the cards the other seven drafters took from the pack that just came back (empty for the first 8 picks of each pack)
`ctx`	`[18]`	facts about the draft, the same for all 45 slots (Section 4.2)
`sid`	scalar	which of the 17 sets this is
`valid`	`[45]`	false for empty slots, so they don't count
`rate`	`[45]`	training-only: how often each pick ended up in the main deck

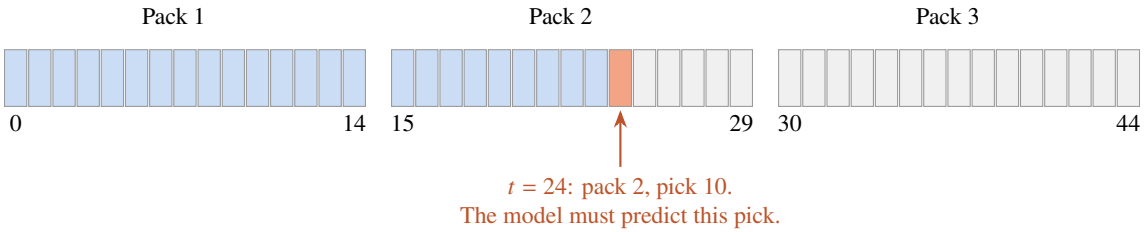


Figure 3: The 45 slots of one draft. When predicting pick $t = 24$ (orange), the drafter has seen every pack up to and including this one, and has made every pick *before* it (blue). Slots after it (grey) are the future and must be hidden.

The model processes 32 drafts at a time (a *batch*), so in the code most tensors have a leading batch dimension $B = 32$: `idx` is `[B, 45, 15]`, and so on. Below we usually drop B and talk about one draft.

One forward pass predicts all 45 picks at once. This is the single most important thing to understand about how the model trains. The model does not run 45 separate times, once per pick. It reads the whole draft in one go and outputs 45 sets of probabilities, one per slot. It can do this without cheating because of the *causal mask* (Section 5): the computation for slot t is wired so that it can only use information from slots $\leq t$ (packs) and $< t$ (picks). Feeding in the real human picks as "the picks made so far" is called *teacher forcing*; it is the same trick language models use to predict every next word of a sentence in parallel.

3 Step 1: turning a card into a vector

Neural networks work on lists of numbers, so the first job is to give every card a vector of $d = 256$ numbers. Think of these as coordinates in a "card space" where similar cards land near each other. A card's vector is the sum of two parts:

1. **A learned vector for the card's name.** A lookup table with one row of 256 numbers per card name (5,094 names). The numbers start random and are tuned during training. Nothing tells the model what they mean; it discovers that, say, one direction in the space means "red" and another means "good in aggressive decks" because that helps it predict picks. Reprints and basic lands share one row across sets, because they are the same card.
2. **A projection of fixed card features.** Each printing (5,317 in all) also has a fixed list of 465 numbers computed from Scryfall: colours, rarity, mana value, card types, power and toughness, keywords, mana production, rules-text roles, and a 384-number text embedding of the rules text (from the small language

model `bge-small-en-v1.5`, with the card's own name blanked out). A learned 465×256 matrix maps these into the same 256-dimensional space.

$$\mathbf{e}_c = \underbrace{\text{NameEmbedding}[\text{name}(c)]}_{\text{learned, per name}} + \underbrace{W_{\text{feat}}\,\mathbf{f}_c}_{\text{learned map of fixed features}} \qquad \mathbf{e}_c \in \mathbb{R}^{256}$$

The features help most when a card has little data, and they give the name vector a sensible starting point. The name vector is where the model stores what the features can't say, chiefly how strong the card actually is in its format. The whole table, `[5318, 256]` (one extra all-zero row for "no card"), is computed once per forward pass and then looked up everywhere a card appears. In the code this is `CardTable`.

A "bag" of cards. Several inputs are a set of cards rather than one card: the pack, the cards the others took. The model turns a set into one vector by taking the *count-weighted average* of the card vectors, and then passing it through a learned 256×256 linear layer. This is what the code calls `bag()`. A pack with two copies of card A and one of card B becomes $\frac{2}{3}\mathbf{e}_A + \frac{1}{3}\mathbf{e}_B$. Averaging throws away order and exact composition, but it keeps the gist: "this pack is mostly green, with one strong rare".

4 Step 2: building one history token per pick

The history encoder reads a sequence of 45 *tokens*, one per pick slot. A token is just a vector of 256 numbers. Token t is the *sum* of six pieces of information, each first mapped into the same 256-number space:

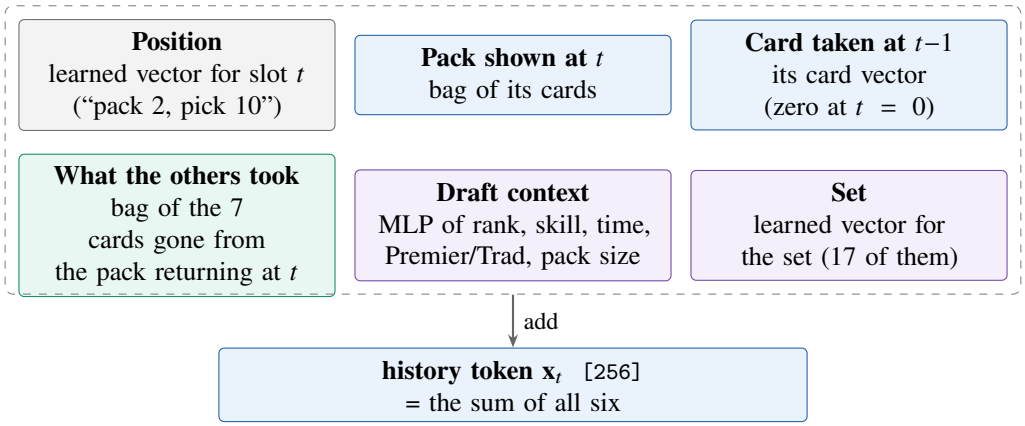


Figure 4: The six ingredients of history token t. Each is mapped to 256 numbers and they are simply added. Adding (rather than concatenating) is standard in transformers: the network learns to keep the ingredients in roughly separate directions of the 256-dimensional space.

- **Position.** One learned vector per slot (45 of them). Without it the model would not know whether a token is pick 1 or pick 14, and the same pack means very different things at those two times.
- **Pack shown at t.** The bag of the cards in the pack in front of the drafter now. Note the asymmetry: token t includes *pack* t but only *pick* $t-1$. The pick at t is what we're trying to predict, so it can't be an input to its own token. (Pick t shows up one token later.)
- **Card taken at $t-1$.** The vector of the card the drafter took last pick. Summed over all earlier tokens by the attention layers, this is how the model knows the drafter's pool.

- **What the others took.** Explained below; this is the input that lets the model read signals.
- **Draft context.** Section 4.2.
- **Set.** A learned vector per set, so the same model can behave differently in different formats (a 3-mana 2/3 is great in one set and filler in another).

4.1 "What the others took"

With eight drafters, a pack you see at pick p comes back to you at pick $p + 8$, after each of the other seven has taken one card from it. Comparing the two snapshots tells you exactly which seven cards the table took, even though the data never says who the other drafters are.

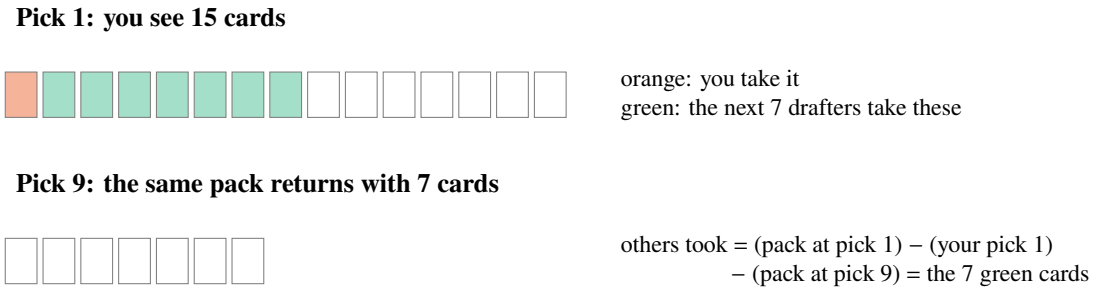


Figure 5: How "what the others took" is computed (`others_taken()` in the code). The same subtraction works for every pick from 9 onward in each pack; for picks 1–8 nothing has come back yet, so this input is zero.

Why it matters: if the green cards you passed are all gone but the blue ones wheel, blue is probably open. Humans call this reading signals. The ablation in the paper finds the whole history (this input plus the attention over earlier packs) adds about 1 point of accuracy, mostly in pack 3.

4.2 Draft context

Eighteen numbers describe the draft as a whole and are identical for all 45 tokens:

feature	numbers	encoding
rank	7	one-hot: bronze, silver, gold, platinum, diamond, mythic, unknown
17Lands win rate	2	centred and scaled, plus a "missing" flag
17Lands games played	2	$\log_{10}$ of games, plus a "missing" flag
days since the format opened	2	linear and log-scaled
calendar time	1	years since 2023
Traditional draft	1	1 for Traditional (best-of-three), 0 for Premier
pack size	3	one-hot: 13, 14 or 15 cards

A small two-layer network (an MLP: $18 \to 256 \to 256$) turns them into a 256-number vector that is added to every token. This is how the model knows Premier from Traditional, mythic from bronze, week 1 from week 8. Time is a continuous number rather than a "which week" lookup because the test drafts come from the end of each format, in weeks no training draft covers; a lookup would have nothing learned for them.

5 Step 3: the history encoder

Now we have 45 tokens, but each one only knows about its own slot. Token 24 holds pack 24 and pick 23, but not pick 5. The history encoder's job is to let every token gather information from the earlier ones, so that the output at slot t summarises the *whole* draft up to t. It does this with *self-attention*.

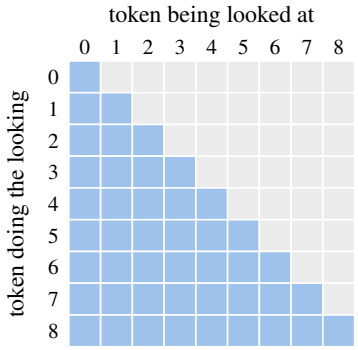
5.1 Attention in one paragraph

Each token computes three vectors from itself: a *query* ("what am I looking for?"), a *key* ("what do I contain?") and a *value* ("what do I pass on if you pick me?"). Token t compares its query with every other token's key; the better the match, the bigger the weight. It then takes the weighted average of the values and adds it to itself. So token 24 might learn to look hard at the tokens where the drafter took red cards, and pull in "I am red" from them. The query, key and value maps are learned. PickNet uses 8 *heads*: 8 independent attention patterns run side by side, each on a 32-number slice of the vector, so one head can track colours while another tracks the curve.

A transformer *layer* is one attention step followed by a small per-token MLP ($256 \to 1024 \to 256$), each wrapped with layer normalisation and a residual ("add the result back to the input") connection. The history encoder stacks **4** of these layers, so information can be gathered and recombined four times.

5.2 The causal mask

If token 24 could attend to token 30, it would see pick 29, which is in the future. The *causal mask* forbids that: token t may attend only to tokens $0, \ldots, t$.



Blue: allowed. Grey: masked out.

Token t contains pack t and pick $t-1$. Attending to tokens $0..t$ therefore gives it **every pack up to and including the current one, and every pick before the current one**: exactly what the human knew when they made pick t.

(The real mask is 45×45; this shows the first 9 slots.)

Figure 6: The causal mask. Because of it, one pass over the whole draft gives 45 honest states, each blind to its own pick and everything after.

The output is one *state* $\mathbf{h}_t$ (256 numbers) per slot, after a final layer normalisation. $\mathbf{h}_t$ is the model's summary of the drafter at the moment of pick t. Because of the mask, it is the same whether you compute it by feeding the full draft or only the first $t+1$ slots; that is what makes the parallel training of Section 2 legitimate, and it is also how you use the model live: feed in the draft so far, read off the last slot.

6 Step 4: candidate tokens

The history encoder knows the drafter; now the model must score the cards in pack t. Each card c in that pack gets its own *candidate token*, again a sum:

$$\mathbf{z}_{t,c} = \underbrace{\mathbf{e}_c}_{\text{card vector}} + \underbrace{\text{CopiesEmbedding}[\text{copies}_c]}_{\text{1, 2 or 3+ copies in the pack}} + \underbrace{W_{\text{state}}\,\mathbf{h}_t}_{\text{the drafter right now}}$$

So the candidate "Card A at pick 24" starts out knowing both what Card A is and who is looking at it. Every pack slot gets up to 15 candidates (fewer as the pack empties), so across the draft there are 45×15 candidate tokens, `[45, 15, 256]`. Empty candidate positions are masked out.

7 Step 5: the pack decoder

The pack decoder refines the candidate tokens with **3 blocks**. Each block does three things, each with layer normalisation before it and a residual connection around it:

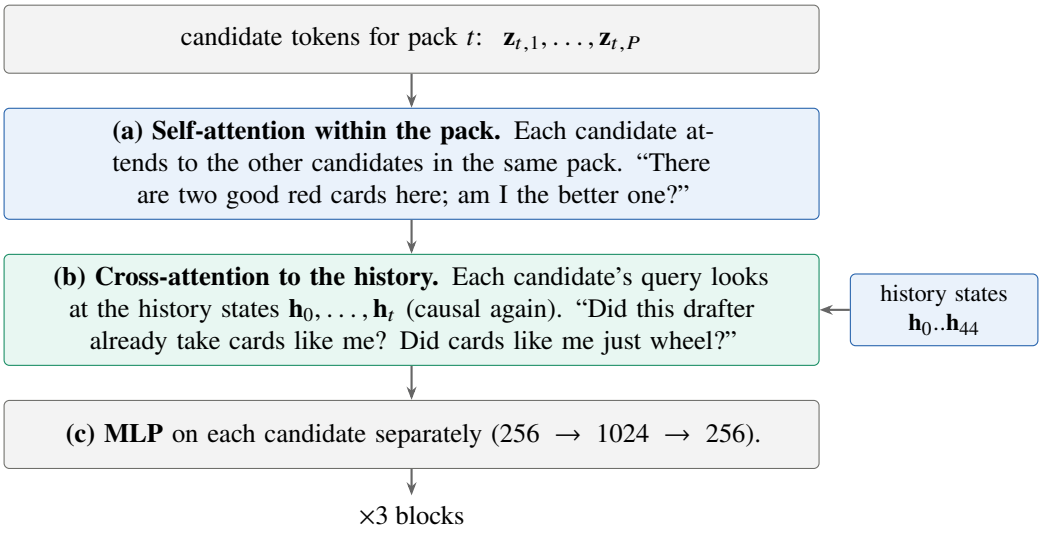


Figure 7: One pack-decoder block (`Block` in the code).

(a) is the step that matters most. A pick-order bot scores each card on its own. But picking is a *comparison*: a decent red card is the right pick when it's the best card in the pack and the wrong one next to a red bomb. Self-attention among the candidates lets each card's score depend on what it's competing against. The paper's ablation credits this step with about 3 points of accuracy, three times what the history of passed packs adds.

(b) differs from what the history encoder did. There, history tokens looked at other history tokens. Here the *candidates* ask the questions and the *history states* answer. The candidate for Card A at pick 24 can pull from any $\mathbf{h}_s$ with $s \le 24$, so it can find, for instance, the moment the drafter moved into a colour.

Self-attention here runs separately within each pack: the candidates of pick 24 never see the candidates of pick 25. That's why the whole thing still fits in one parallel pass.

8 Step 6: from scores to a pick

After the last block, a small head (layer norm, 256 → 256, GELU, 256 → 1) turns each candidate token into a single number, its *logit* (score). The probability of taking card c is the softmax over the cards actually in

the pack:

$$P(\text{take } c \mid \text{everything so far}) \;=\; \frac{\exp(\text{logit}_c)}{\sum_{c' \in \text{pack}_t} \exp(\text{logit}_{c'})}$$

Cards not in the pack aren't candidates, so they get no probability at all. This "softmax over only the available options" is the same as the conditional logit model economists use for consumer choice.

A worked example. Suppose pack t has five cards left and the model outputs these logits. (The cards are made up for illustration.)

card	logit	exp(logit)	probability	
A: on-colour removal spell	2.1	8.17	0.574	
B: on-colour 2-drop	1.3	3.67	0.258	
C: off-colour rare	0.4	1.49	0.105	
D: on-colour filler	−0.5	0.61	0.043	
E: off-colour filler	−1.2	0.30	0.021	
sum		14.24	1.000	

- **Top-1 accuracy**: the model's prediction is A. If the human took A, this pick counts as correct. The paper's 70.8% is the share of test picks where this happens.
- **Loss (NLL)**: if the human took B, the loss for this pick is $-\ln 0.258 = 1.35$ nats; had they taken A, it would be $-\ln 0.574 = 0.56$. Training lowers the average of this number over all valid picks. Unlike top-1, it rewards being confidently right and punishes being confidently wrong, which is why the paper reports both. The model averages 0.749 nats per pick on the test set.

Forced picks (the last card of a pack) have one candidate, so their probability is always 1 and their loss 0. That's why the paper also reports accuracy on non-forced picks.

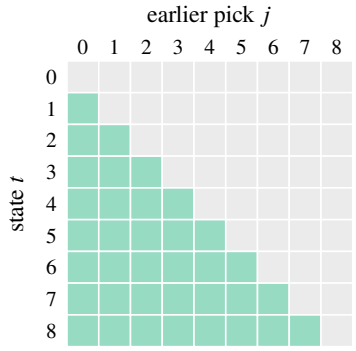
9 The deck head: a training aid

Alongside the pick loss, the model gets a second, smaller job, borrowed from the puder model: from the state $\mathbf{h}_t$, predict for every earlier pick $j < t$ whether that card ends up in the drafter's main deck.

$$\text{deck logit}_{t,j} \;=\; \frac{(W_q \mathbf{h}_t) \cdot (W_k \mathbf{e}_{\text{pick}_j})}{\sqrt{256}}$$

This is a dot product between "the drafter at time t" and "the card picked at j". The target is the card's 17Lands main-deck rate for that drafter (a number between 0 and 1), scored with binary cross-entropy. Later states get more weight (weight $(t+1)/45$), since by pack 3 the drafter's deck is much clearer than at pick 2. This loss is multiplied by 0.3 and added to the pick loss.

Why bother? Knowing which of your picks will make the deck is close to knowing which archetype you are in, and that is exactly what the history state needs to encode for good pick predictions. The extra target pushes the history encoder toward that knowledge. The deck head is never used to make a pick, and the paper notes it was not ablated, so how much it helps is unmeasured.



Green: scored. State t has seen picks $0..t-1$, so it predicts the deck fate of those picks only (strictly below the diagonal; note this differs from the attention mask, which includes the diagonal).

Figure 8: Which (state, pick) pairs the deck head is trained on.

10 Training

- **Data.** 1.72M training drafts from 17 sets, Premier and Traditional, shuffled together. Each batch is 32 drafts, so $32 \times 45 = 1{,}440$ picks contribute to each update (minus empty slots).
- **Loss.** Mean pick NLL over valid picks + 0.3× deck loss.
- **Optimiser.** AdamW (weight decay 0.01), gradients clipped at norm 1, a one-cycle learning-rate schedule: a short warm-up, then a cosine decay to near zero. One cycle is 53,791 steps, one pass over the training drafts.
- **Two epochs.** Epoch 1 peaks at learning rate 5×10^{-4}. Epoch 2 starts from epoch 1's best weights, peaks at 2×10^{-4} and sees the drafts in a new order.
- **Early stopping.** Every 2,500 steps the model is scored on 4,000 held-out Premier drafts; the best-scoring weights are kept.
- **Regularisation.** Dropout 0.05 throughout. The "pack dropout" option (hide the packs from the history for half the drafts) exists in the code but is off in the final model; it made no difference.

11 Shapes and sizes

Tensor shapes through one forward pass, for a batch of $B = 32$ drafts ($T = 45$ slots, $P = 15$ candidate positions, $d = 256$):

stage	shape	code
card table	`[5318, 256]`	`tab = self.cards()`
pack bag, pick vector, others bag, position	`[B, 45, 256]` each	`bag(...)`, `prev`, `self.pos`
context, set	`[B, 1, 256]`, broadcast to all 45	`ctx_mlp`, `set_emb`
history tokens **x**	`[B, 45, 256]`	`h` before `self.hist`
history states **h**	`[B, 45, 256]`	`h` after `hist_norm`
candidate tokens **z**	`[B, 45, 15, 256]`	`x`
(a) self-attention	over the 15 within each of $B \cdot 45$ packs	`Block`, first half
(b) cross-attention	$45 \cdot 15$ queries per draft over 45 states	`Block`, second half
logits	`[B, 45, 15]`	`self.head(x)`
deck logits	`[B, 45, 45]`	`deck_logits()`

Where the 8.36M parameters are:

component	parameters	share
card table (5,094 name vectors + feature projection)	1,423,360	17%
history encoder (4 transformer layers + final norm)	3,159,552	38%
pack decoder (3 blocks)	3,160,320	38%
token input layers (pack, pick, others, context, position, set)	283,904	3%
candidate projection, copies, scoring head	133,377	2%
deck head	131,584	2%
unused (pool layer for the no-history ablation)	65,792	1%
total	8,357,889	

12 Putting it together: one pick, start to finish

Say we want the model's prediction for a mythic-rank Premier drafter at pack 2, pick 10 ($t = 24$).

1. **Look up card vectors** for every card that appears anywhere in the draft so far.
2. **Build tokens 0–24.** Token 24 = position 24 + bag(the 6 cards in front of them) + vector of the card they took at pick 9 of pack 2 + bag(the 7 cards the table took from this pack since they saw it at pick 2) + context(mythic, Premier, day 23 of the format, …) + the set's vector.
3. **Run the history encoder.** Four layers of causal attention. State $\mathbf{h}_{24}$ now reflects all 24 earlier picks (their pool), every pack they saw, and every returning pack.
4. **Build 6 candidate tokens**, one per card in the pack: card vector + copies + projected $\mathbf{h}_{24}$.
5. **Run the pack decoder.** In each of 3 blocks the six candidates compare themselves with each other, consult $\mathbf{h}_0..\mathbf{h}_{24}$, and pass through an MLP.
6. **Score.** Six logits, a softmax over those six, and the highest-probability card is the prediction.

In training, steps 1–6 happen for all 45 slots and all 32 drafts in the batch at once, and the losses from all valid picks are averaged before each update.

13 FAQ

Why not just feed the model "the cards in my pool" and "the cards in the pack"? That's roughly the *pool* baseline in the paper: a pick-order rating per card plus a learned card-by-pool match score. It gets 64.4% on MSH. Adding the within-pack comparison gets to 67.3%, and the full history to 68.2%. The transformer machinery is what buys those points.

Does the model know who the other drafters are? No. It only sees which cards vanished from the packs that came back. It has to infer what the neighbours are drafting from that.

How does one model handle 17 sets? Every set's cards live in one shared card table (reprints share a vector), the set embedding tells the model which format it's in, and card features give new cards a sensible starting point. Pooling the sets added 1.3 points over the best single-set model on MSH.

Does it know Premier from Traditional? Yes, through the Traditional flag in the draft context (Section 4.2). It trains on both but is validated and tested on Premier only.

Can it draft a set it has never seen? Not well. The name vectors for new cards would be untrained. A features-only variant (no name vectors) trained on 7 other sets scores 50.8% on unseen MSH, barely above a pick-order bot fitted on MSH itself.

Is it drafting *well*? It imitates 17Lands users, who are above average but not optimal. A high top-1 means "picks like these humans", not "picks the best card".